

EE446-TinyML Lab 6

Author: Sparsh Dadhich

Github: https://github.com/sdadhich04/EE-446-TinyML-/tree/main/Lab6_TinyML_Anomaly_Detection

Edge Impulse: <https://studio.edgeimpulse.com/public/997921/live>

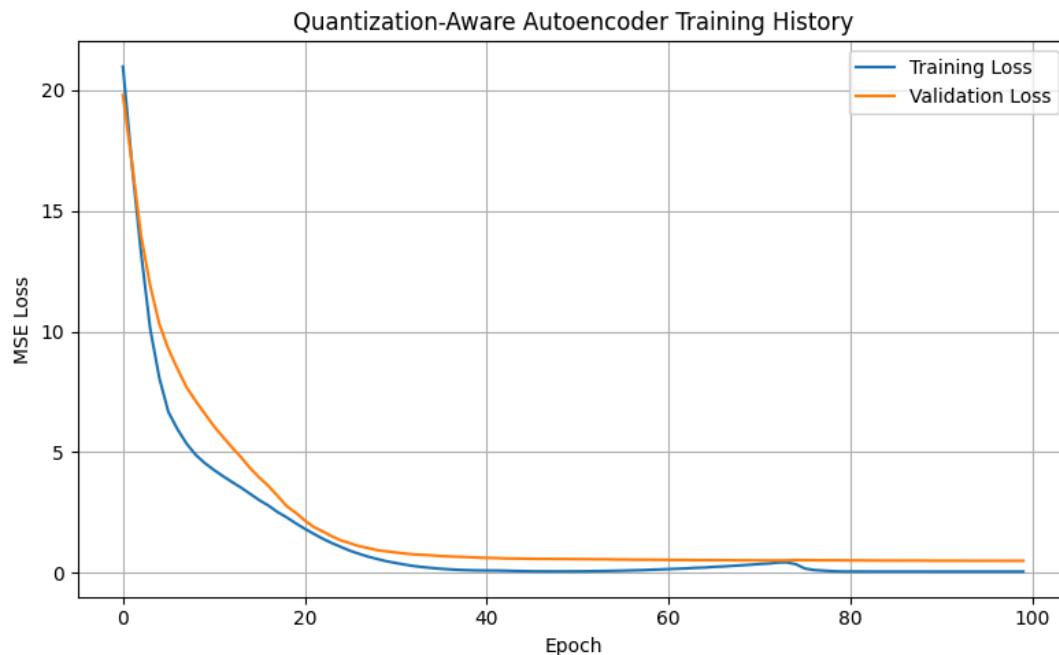
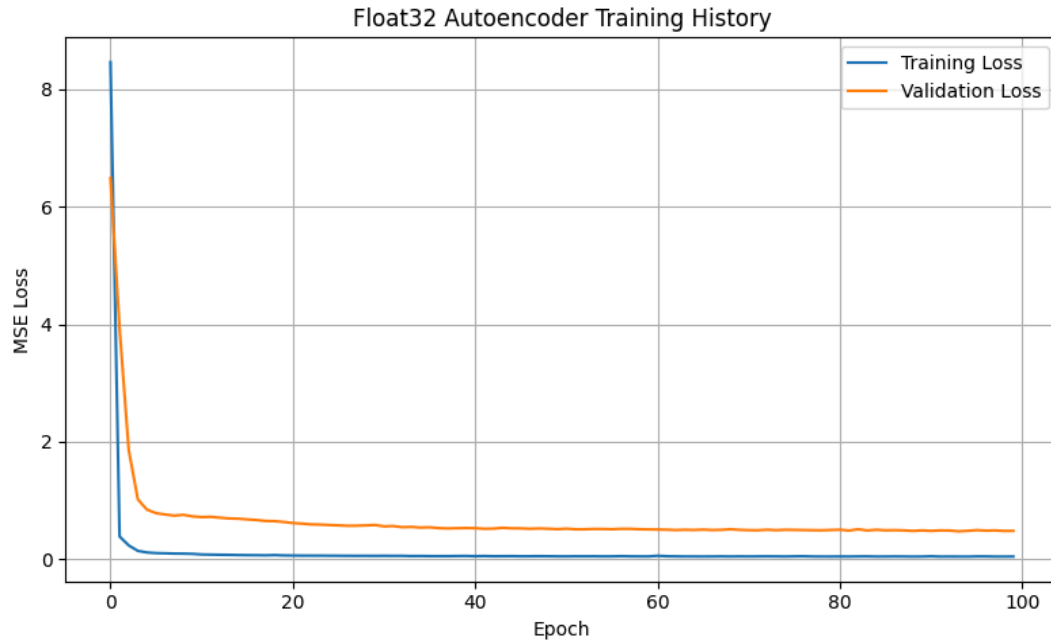
Part 1

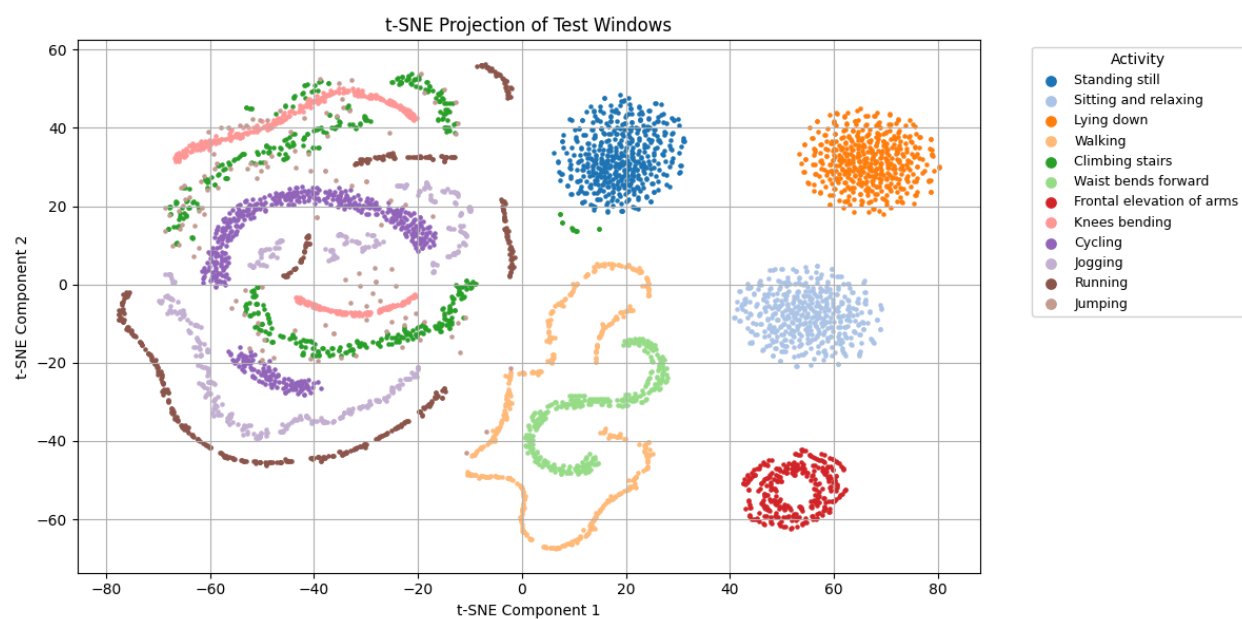
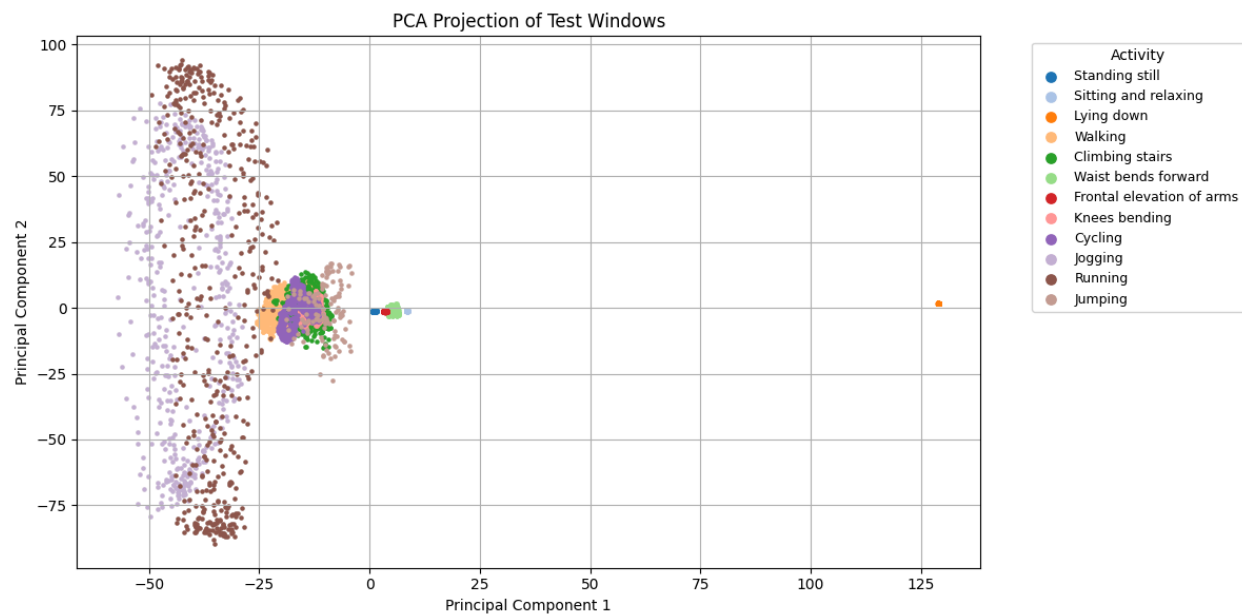
1. The Edge Impulse project used raw accelerometer data from the Arduino Nano 33 BLE Sense Rev2, specifically the accX, accY, and accZ axes. The impulse was configured with a 2000 ms window size, 220 ms stride, and 100 Hz sampling frequency to capture time-series motion data for activity classification and anomaly detection.
2. The neural network classifier used 33 spectral analysis features extracted from the accelerometer signals. These features included RMS values, spectral power, peak frequencies, and peak heights. Example feature names included accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Freq, and accY RMS. These frequency-domain features allowed the model to distinguish between different motion patterns more effectively than raw sensor values alone.
3. The classifier produced two outputs: nominal and off. The model architecture contained an input layer with 33 features, dense layers with 32 and 16 neurons, and a 2-class output layer. The quantized int8 model achieved 100% validation accuracy with a loss of 0.01 during training.
4. The anomaly detector used spectral features such as accX RMS, accX Peak 1 Height, accY Spectral Power 2.0–5.0 Hz, and accZ Spectral Power 2.0–5.0 Hz. A K-means clustering model with 32 clusters was trained to learn normal feature distributions and identify unusual motion patterns during inference.

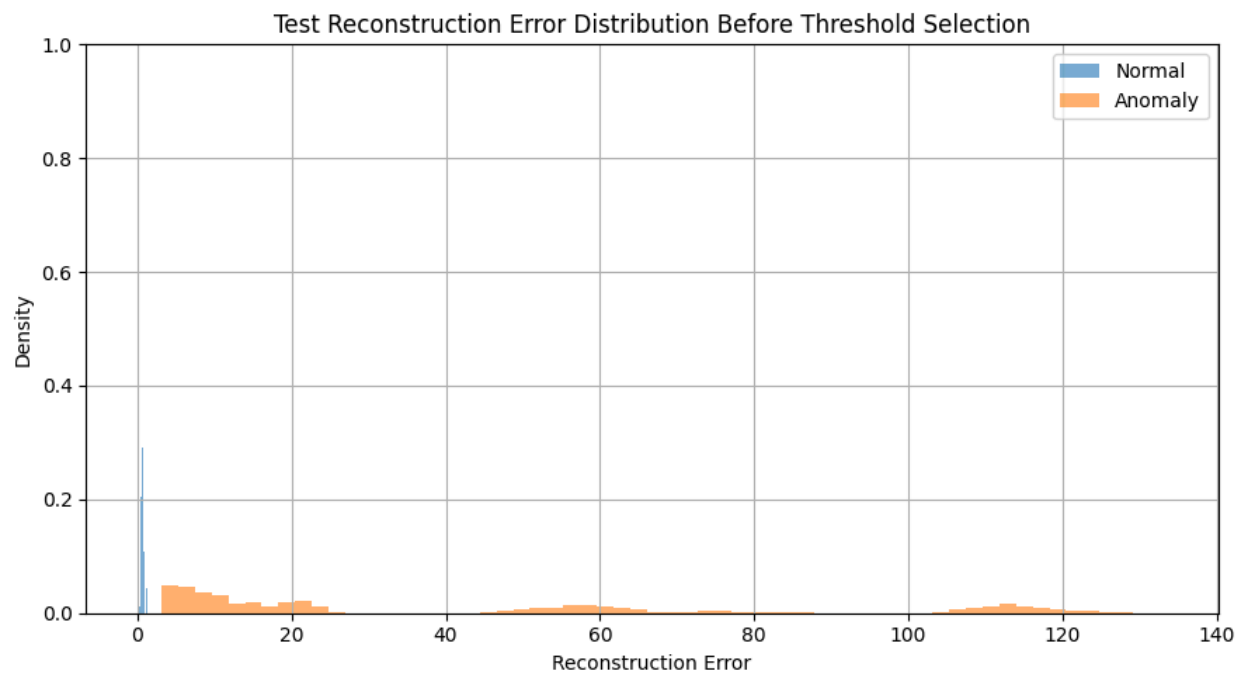
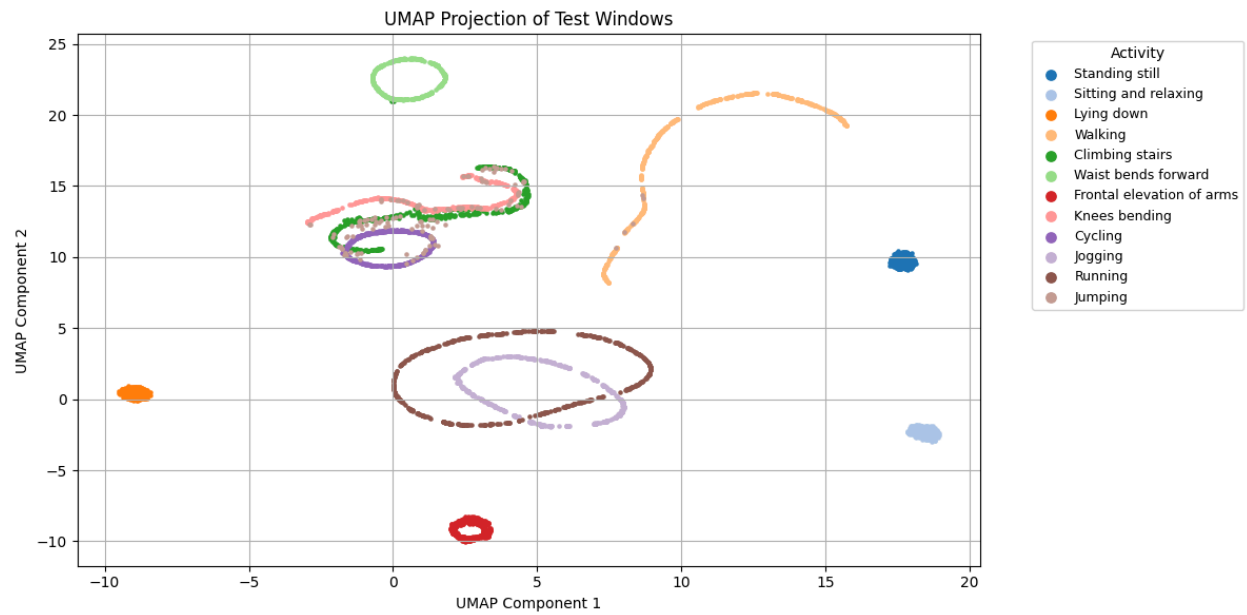
```
nominal: 0.437500
off: 0.562500
Anomaly prediction: 16.013824
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 611 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.019449
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 610 us, anomaly 509 us, postprocessing 50 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.023281
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 609 us, anomaly 6 ms, postprocessing 67 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.021883
Starting inferencing in 2 seconds...
Sampling...
```

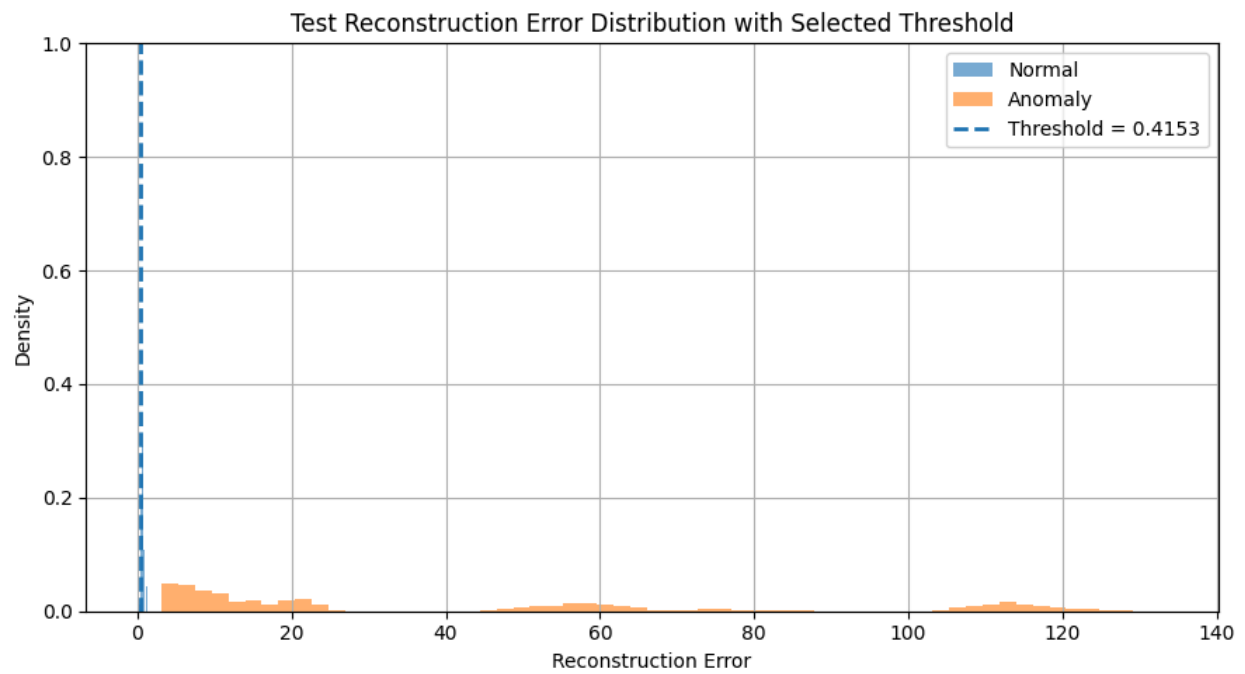
6. The deployment results showed clear separation between nominal and abnormal motion patterns, allowing accurate real-time classification and anomaly detection. However, classifier outputs should not always be fully trusted because factors such as sensor noise, unseen motion patterns, environmental variation, and threshold selection can reduce reliability and produce false predictions. Despite these limitations, the project demonstrated that TinyML can perform lightweight and effective embedded inference directly on low-power hardware.

Part 2

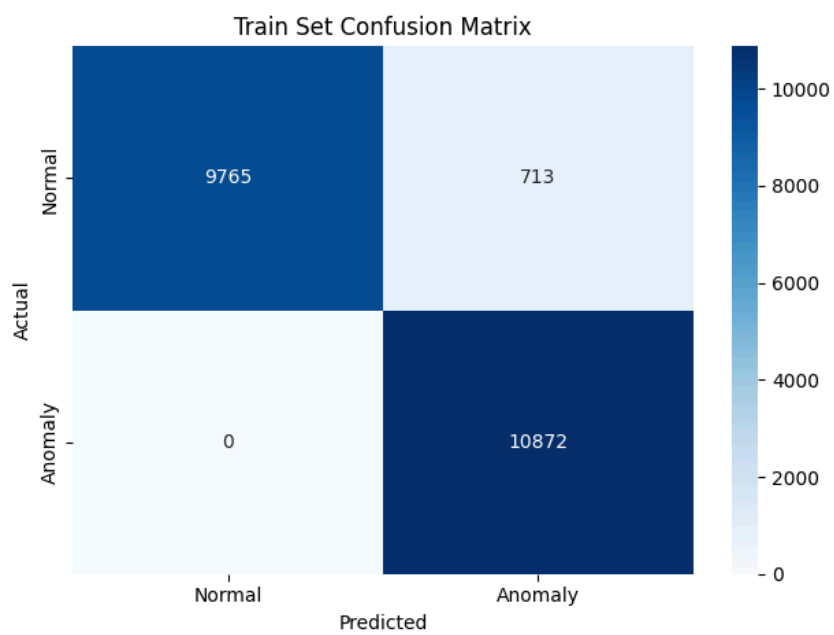


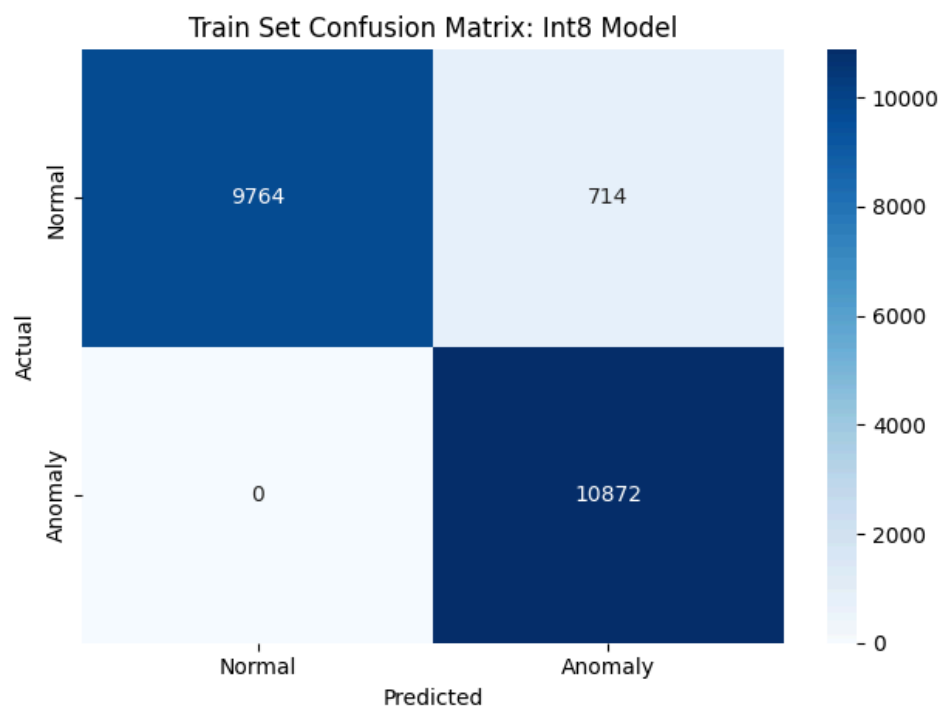
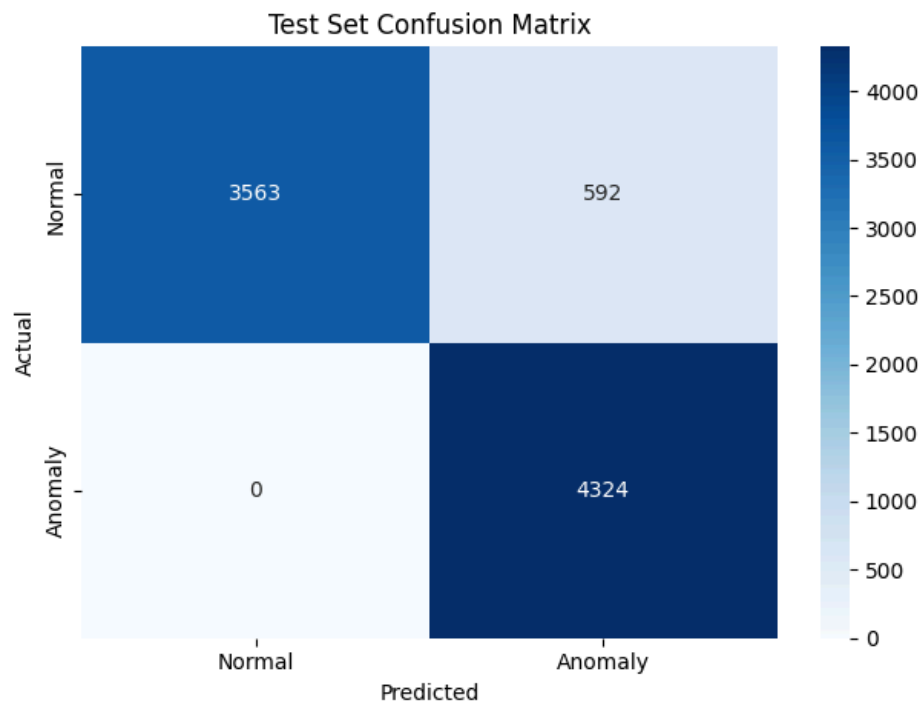


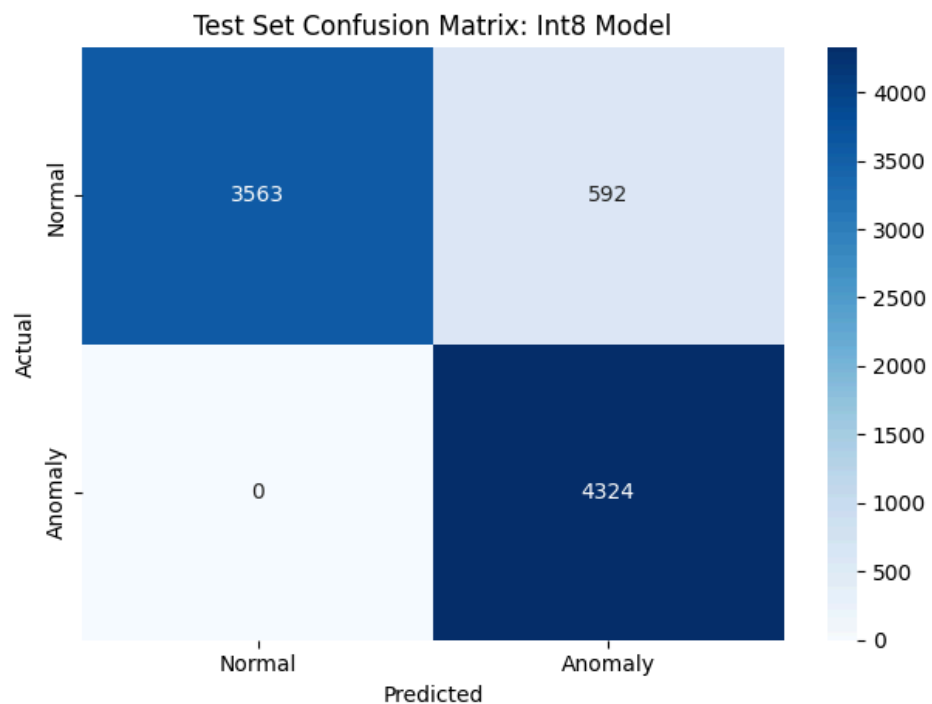




Threshold = 0.4152873009443283







Train set evaluation

	precision	recall	f1-score	support
Normal (0)	1.00	0.93	0.96	10478
Anomaly (1)	0.94	1.00	0.97	10872
accuracy			0.97	21350
macro avg	0.97	0.97	0.97	21350
weighted avg	0.97	0.97	0.97	21350

Test set evaluation

	precision	recall	f1-score	support
Normal (0)	1.00	0.86	0.92	4155
Anomaly (1)	0.88	1.00	0.94	4324
accuracy			0.93	8479
macro avg	0.94	0.93	0.93	8479
weighted avg	0.94	0.93	0.93	8479

Train set evaluation: int8 model

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Normal (0)	1.00	0.93	0.96	10478
Anomaly (1)	0.94	1.00	0.97	10872
accuracy			0.97	21350
macro avg	0.97	0.97	0.97	21350
weighted avg	0.97	0.97	0.97	21350

Test set evaluation: int8 model

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Normal (0)	1.00	0.86	0.92	4155
Anomaly (1)	0.88	1.00	0.94	4324
accuracy			0.93	8479
macro avg	0.94	0.93	0.93	8479
weighted avg	0.94	0.93	0.93	8479

```
Reconstruction error: 0.408825
Result: Normal activity
Reconstruction error: 0.416903
Result: Normal activity
Reconstruction error: 0.416909
Result: Normal activity
Reconstruction error: 0.416804
Result: Normal activity
Reconstruction error: 0.416701
Result: Normal activity
Reconstruction error: 0.416571
Result: Normal activity
Reconstruction error: 0.416438
Result: Normal activity
Reconstruction error: 0.416367
Result: Normal activity
Reconstruction error: 0.416244
Result: Normal activity
```

```
Reconstruction error: 1.942497
Result: Anomaly detected
Reconstruction error: 2.058063
Result: Anomaly detected
Reconstruction error: 2.453112
Result: Anomaly detected
Reconstruction error: 3.203115
Result: Anomaly detected
Reconstruction error: 2.918711
Result: Anomaly detected
Reconstruction error: 2.545860
Result: Anomaly detected
Reconstruction error: 2.569746
Result: Anomaly detected
Reconstruction error: 2.711610
Result: Anomaly detected
Reconstruction error: 2.853038
```


Part 3

1. The anomaly detection threshold selected was approximately 0.4153, calculated using the mean reconstruction error of the normal training data plus two standard deviations. This threshold was chosen because it provided a good balance between detecting anomalous activities while minimizing false positives on normal activities.
2. Reconstruction error measures how well the autoencoder can recreate the original input signal. Since the model was trained primarily on normal activity patterns, it learned to reconstruct normal motions with low error. When the model encounters unfamiliar or abnormal motion patterns, the reconstruction becomes less accurate, resulting in a much higher reconstruction error. This difference allows the system to distinguish between normal and anomalous activities during inference.
3. Several important parameters from the Python notebook must be preserved during deployment, including the trained quantized .tflite model, the selected anomaly threshold, input window size, feature ordering, quantization parameters, and preprocessing assumptions. These values ensure that the Arduino performs inference using the same conditions under which the model was trained.
4. The Arduino sketch must use the same preprocessing assumptions because the model expects input data in exactly the same format used during training. Differences in sampling frequency, window size, normalization, feature ordering, or sensor scaling could change the input distribution and significantly reduce model accuracy. Consistent preprocessing ensures that the deployed model receives data that matches the learned feature patterns from the notebook training process.
5. The main limitations of this anomaly detection method include limited computational resources, restricted memory, sensor noise, and reduced precision due to quantization. The model may also struggle with unseen motion patterns, different users, or environmental variations that were not present in the training dataset. Additionally, threshold selection can strongly affect performance, potentially causing false positives or missed anomalies during real-time deployment on embedded hardware.